Emulazione del daemon `httpd` (Tenda AC15) tramite Qiling

Indice

Introduzione e Architettura del Sistema di Emulazione **2**

Analisi dei Problemi Riscontrati, Origini e Soluzioni **2**

Anomalie e Crash Interni del Motore Qiling 2

Crash di Asserzione su Syscall con Lunghezza Zero (`is_mapped`) 2

Crash del Thread Manager non Inizializzato (`cur_thread` = `None`) e Gestione di `clone` 2

Crash sulla Chiusura delle socket (`shutdown` con `Errno 107: ENOTCONN`) 3

Comunicazione Inter-Processo (IPC) e Blocchi Apparenti (Hang) 3

Loop Infinito di Polling sulle Code di Messaggi System V (`ftok` e `msgget`) 3

File System Virtuale, Linker Dinamico e Configurazione di Rete 4

Mancanza di File e Directory Volatili e Rimozione di `core_pattern` 4

Sottosistema di Tracciamento, Hook e Logging Multi-Processo 5

Architettura di Logging Multi-Processo e Cattura di `stdout`/`stderr` (`emu_logging.py`) 5

Sottosistema per il Debugging (`gdb_debugging_ext.py` / `emu_gdb.py`) 6

Colli di Bottiglia del Server GDB Nativo di Qiling e generale mancanza di feature QoL(`ql.debugger = "gdb"`) 6

Limitazioni Attuali **8**

Assenza dei deamon riceventi e Semantica IPC Incompleta 8

Assenza di reale multithreading 8

Introduzione e Architettura del Sistema di Emulazione

L'obiettivo di questo progetto è costruire (o quantomeno gettare le basi) per un ambiente di emulazione per il firmware del router Tenda AC15 AC1900 (versione 15.03.05.19), affetto da un'ampissimo range di vulnerabilità (ad oggi più di 85 CVE), al fine di eseguire live analysis di esse attraverso `gdb`.

L'emulazione diretta

L'environment di emulazione è articolato nei seguenti script Python:

- `emu_gdb.py`: Script principale contenente l'inizializzazione del sandbox Qiling, il thread di ascolto per il socket NVRAM (`nvram_listener`), gli hook sulle system call POSIX e l'aggancio ai trigger di `GdbManager`.

- `gdb_debugging_ext.py`: Modulo che implementa la classe `GdbManager` per il debugging basato su eventi, la trappola sui memory fault, il trace sui basic block, la generazione automatica degli script `attach_<pid>.gdb` con risoluzione dell'offset `.text` delle librerie ELF e i comandi `monitor` via pacchetti `qRcmd`.

- `ipc_shim.py`: Modulo che sostituisce l'implementazione interna delle code di messaggi System V di Qiling con chiamate dirette al kernel dell'host Linux tramite `ctypes`.

- `emu_logging.py`: Infrastruttura di logging duale (`complete_log` e `small_log`) con granularità differenti e tracciamento del PID host reale.

Analisi dei Problemi Riscontrati, Origini e Soluzioni

Di seguito vengono esaminati i principali problemi riscontrati utilizzando Qiling out-of-the-box, classificati per area funzionale.

Anomalie e Crash Interni del Motore Qiling

Crash di Asserzione su Syscall con Lunghezza Zero (`is_mapped`)

- **Problema riscontrato:** Nelle prime fasi di avvio, l'emulatore terminava improvvisamente all'interno di `ql_syscall_write` sollevando un'eccezione `AssertionError: expected a positive size value`.

- **Origine del problema:** Il binario emulato invoca legittimamente la system call POSIX `write(fd, buf, 0)`. Prima di eseguire l'operazione, Qiling verifica che il buffer sia mappato in memoria attraverso `ql.mem.is_mapped(buf, count)`, il quale contiene un'asserzione interna che impone `size > 0`.

- **Soluzione implementata:** Inizialmente è stata adottata una soluzione parziale consistente in un hook per la sola syscall `write` che riceveva silenziosamente l'eccezione, successivamente è stata adottata una soluzione più generale ed efficace patchando direttamente il metodo `ql.mem.is_mapped`. La funzione wrapper `safe_is_mapped(addr, size, ...)` verifica se `size <= 0` e, in tal caso, restituisce immediatamente `True`, aggirando l'asserzione per qualsiasi syscall che operi su buffer di lunghezza nulla.

Crash del Thread Manager non Inizializzato (`cur_thread` = `None`) e Gestione di `clone`

- **Problema riscontrato:** L'esecuzione della system call `clone` e l'accesso agli hook usati per leggere il traffico di rete causavano il crash dell'emulatore con errore `AttributeError: 'NoneType' object has no attribute 'cur_thread'` (o `'id'`).

- **Origine del problema:** Su architettura ARM o in determinate fasi di inizializzazione single-threaded, il sottosistema `ql.os.thread_management` di Qiling mantiene l'attributo `cur_thread` impostato a `None`. Quando `ql_syscall_clone` tenta di leggere l'ID del thread corrente per duplicarne lo stato, l'accesso all'attributo fallisce.

- **Soluzione implementata (Discrepanza README vs Codice):** Inizialmente è stato tentato il reindirizzamento forzato di `clone` verso `ql_syscall_vfork`. Tuttavia, poiché in Linux `clone` gestisce sia la creazione di processi che di thread (`pthread_create` con flag `CLONE_VM`), l'uso forzato `vfork` rompeva il multithreading nativo, bloccando il processo padre. Nel codice definitivo è stata invece implementata la funzione `high_fidelity_clone`:

  1. Controlla se `tm.cur_thread` è `None`.
  2. In caso affermativo, istanzia un oggetto mock temporaneo `TempThread` il cui attributo `id` corrisponde con il PID del processo (`getattr(ql.os, 'pid', 1000)`).
  3. Invoca `sched.ql_syscall_clone(ql, *args)`, preservando il parsing dei flag e il mapping TLS (Thread Local Storage).
  4. Nel blocco `finally`, ripristina `tm.cur_thread = None` per non corrompere lo scheduler durante il context switching.

  Parallelamente, per gli hook di rete e `stat` è stata scritta la funzione helper `_get_tid(ql)`, che cattura `AttributeError` e ripiega su `getattr(ql.os, 'pid', 0)`.

Crash sulla Chiusura delle socket (`shutdown` con `Errno 107: ENOTCONN`)

- **Problema riscontrato:** Al termine di una transazione HTTP o alla caduta di una connessione, Qiling crashava dentro `ql_syscall_shutdown` sollevando `OSError: [Errno 107] Transport endpoint is not connected`.

- **Origine del problema:** Il binario `httpd` invoca `shutdown()` su un file descriptor di una socket che è già stata disconnessa dal peer. Il kernel Linux host restituisce l'errore `ENOTCONN` (107), che Python traduce in un'eccezione `OSError` non catturata dall'implementazione standard di Qiling.

- **Soluzione implementata:** La syscall `shutdown` è stata sostituita con `my_shutdown(ql, sockfd, how)`. La funzione recupera l'oggetto socket tramite `get_opened_socket(ql.os, sockfd)`, esegue `sock.shutdown(how)` all'interno di un blocco `try-except OSError`, registra un warning sul logger dedicato (`log_sock`) e restituisce al binario guest il codice di errore negativo conforme allo standard Linux (`-error_number`, ovvero `-107`).

Comunicazione Inter-Processo (IPC) e Blocchi Apparenti (Hang)

Loop Infinito di Polling sulle Code di Messaggi System V (`ftok` e `msgget`)

- **Problema riscontrato:** Il processo `httpd` e i processi figli generati (`sh`, `cfm`) sembravano bloccarsi indefinitamente in chiamate `wait4`, `select` o `nanosleep`, terminando dopo circa 30 secondi con i messaggi `PostMsg msg create error` e `Post Msg failed`.

- **Origine del problema:** La funzione `PostMsg()` del firmware (e il comando `cfm post <modulo>`) calcola la chiave IPC tramite `key = ftok("/var/<modulo>", 0)` (es. `/var/netctrl`, `/var/time_check`) e successivamente chiama `msgget(key, IPC_EXCL | 0666)` **senza** specificare il flag `IPC_CREAT`, in quanto questa viene creata da un altro daemon nel firmware originale. Il blocco aveva una doppia origine:

1. *Assenza del file target per `ftok`:* La funzione di libreria `ftok()` esegue una `stat()` sul percorso per ricavare l'inode. Non esistendo il file in `/var/`, `stat()` falliva, `ftok()` restituiva `-1` (`0xffffffff`) e `msgget` falliva.
2. *Inadeguatezza delle code interne di Qiling:* Anche con una chiave valida, nessun demone esterno era in esecuzione per creare la coda. Inoltre, le code native di Qiling risiedono in un dizionario Python (`ql.os.msq`): poiché Qiling emula `fork()` invocando la vera `os.fork()` del sistema operativo host, ogni processo figlio eredita una copia privata e isolata del dizionario, rendendo impossibile la comunicazione tra processi distinti. Infine, la `msgrcv()` nativa di Qiling esegue un busy-spin in Python.

- **Soluzione implementata (`ipc_shim.py`):**

1. *Tracciamento `stat`+`msgget`:* In `emu_gdb.py` sono stati inseriti hook di ingresso e uscita su `stat`, `stat64`, `lstat` e `lstat64` per correlare i fallimenti di `stat` con le chiamate `msgget(key=0xffffffff)`.
2. *Delega al Kernel Host (`IpcShim`):* Le syscall `msgget`, `msgsnd` e `msgrcv` sono state rimpiazzate nel modulo `ipc_shim.py` (attivo di default a meno che `EMU_IPC=off`) con wrapper che invocano direttamente le funzioni della `libc` dell'host Linux tramite `ctypes`. Le code del kernel host sono nativamente condivise tra tutti i processi generati da `os.fork()`.
3. *Creazione On-Demand (`force_create=True`):* Quando il binario chiama `msgget` senza `IPC_CREAT`, lo shim tenta comunque di creare la coda passando `IPC_CREAT | IPC_EXCL | mode`, sbloccando immediatamente il mittente.
4. *Prevenzione dello stallo in scrittura (`drop_oldest_when_full=True`):* Poiché i demoni riceventi non sono attivi, le code si riempirebbero bloccando i mittenti. Lo shim forza `IPC_NOWAIT` fino a 64 tentativi: se `msgsnd` fallisce con `EAGAIN`, rimuove automaticamente il messaggio più vecchio tramite una `msgrcv(..., 8192, 0, IPC_NOWAIT | MSG_NOERROR)` e ritenta l'invio.
5. *Polling non bloccante in lettura (`msgrcv`):* Per evitare di congelare l'intero emulatore in un'attesa kernel non interrompibile da `Ctrl-C`, `IpcShim.msgrcv` interroga sempre il kernel host con `IPC_NOWAIT` e, in caso di `ENOMSG` su chiamata bloccante, attende in Python con `time.sleep(0.01)` fino all'eventuale scadenza di `rcv_timeout`.
6. *Traduzione dell'ABI (32-bit vs 64-bit):* La struttura `struct msgbuf { long mtype; char mtext[]; }` utilizza un `long` di 4 byte sul guest ARM32, ma di 8 byte sull'host x86-64. Lo shim converte dinamicamente il campo `mtype` e l'offset del payload ad ogni invio e ricezione.
7. *Pulizia automatica:* Ogni coda creata sull'host viene annotata nel file `logs/ipc_queues.txt` nel formato `msqid key creator_pid` e rimossa all'uscita dal solo processo padre (`os.getpid() == self._owner_pid`) tramite un handler `atexit` (utilizzando `msgctl(msqid, IPC_RMID, None)`), oltre a ripulire all'avvio eventuali code orfane di esecuzioni precedenti.

File System Virtuale, Linker Dinamico e Configurazione di Rete

Mancanza di File e Directory Volatili e Rimozione di `core_pattern`

- **Problema riscontrato:** Errori `ENOENT` (File o directory non esistente) durante l'apertura di file di stato o di configurazione temporanei in `/var`, `/tmp`, `/etc`, `/dev`, `/proc` e `/sys`.

- **Origine del problema:** L'estrazione statica del file system SquashFS non contiene le directory e i file generati dinamicamente in RAM (`tmpfs`) durante la sequenza di boot reale del router.

- **Soluzione implementata:** La problematica viene risolta in due momenti separati:

  1. All'avvio di `my_sandbox`, vengono create direttamente dentro `ROOTFS` le directory essenziali (`dev`, `proc`, `sys`, `tmp`, `var/run`), viene mappato `/dev/urandom` sull'equivalente host e viene rimosso l'eventuale file `{ROOTFS}/proc/sys/kernel/core_pattern`.
  2. La funzione `auto_create_virtual_file(path)` opera direttamente su `ROOTFS` (escludendo i file `.so` o contenenti `.so.` per non interferire con la ricerca dei percorsi del dynamic linker) ed è invocata **esclusivamente** dagli hook di ingresso di `open` (`hook_open_enter`) e `openat` (`hook_openat_enter`, limitatamente ai path assoluti che iniziano con `'/'`), e **non** durante le chiamate `stat`. Inoltre, la funzione helper `_o_creat(ql)` distingue dinamicamente il flag `O_CREAT` tra architettura MIPS (`0x100`) e ARM/x86 (`0x40`, dove `0x100` è invece `O_NOCTTY`) per evitare falsi positivi nei log di creazione file.

Sottosistema di Tracciamento, Hook e Logging Multi-Processo

Architettura di Logging Multi-Processo e Cattura di `stdout`/`stderr` (`emu_logging.py`)

- **Problema riscontrato:**

  1. Impossibilità di distinguere i processi nei log: `ql.os.pid` restituiva sempre lo stesso valore statico emulato (es. `1996`) per tutti i processi figli generati da `fork()`, e il campo `%(process)d` del modulo `logging` di Python produceva `None` causando errori di formattazione.
  2. I messaggi stampati dal binario guest su `stdout` e `stderr` venivano inviati direttamente al terminale host, aggirando completamente il logger di Qiling.
  3. Gli errori interni agli hook venivano silenziati da blocchi `except: pass`.

- **Origine del problema:** Qiling imposta globalmente `logging.logProcesses = False` e crea per ogni istanza un logger denominato `qiling<id>` con `propagate = False`.

- **Soluzione implementata:**

  1. Nel modulo `emu_logging.py` è stato definito il formatter `_PidFormatter`, che inietta dinamicamente il vero PID del sistema operativo host (`os.getpid()`) nell'attributo `record.hostpid` di ogni record di log al momento della formattazione, garantendo il tracciamento esatto anche dopo una `os.fork()`. L'hook `hook_execve` legge fino a 16 argomenti da `argv[]` (rispettando `ql.arch.pointersize`) e registra sia `emu_pid` sia `host_pid` e `host_ppid`.
  2. Il metodo `logs.attach_qiling(ql)` collega direttamente il file handler di `complete_log` (livello `DEBUG`, configurabile in verbosità tramite `EMU_QL_VERBOSE=off|default|debug|disasm`) all'istanza `ql.log`, interlacciando in ordine cronologico reale le tracce interne di Qiling con gli eventi degli hook `emu.hook.<nome>` (che al livello `INFO` confluiscono invece anche in `small_log` e su console).
  3. È stato aggiunto `hook_write_enter` sui file descriptor `1` (`stdout`) e `2` (`stderr`) (per scritture fino a 4096 byte) per catturare l'output testuale del binario guest e convogliarlo nel logger `guest_stdio`.
  4. Negli hook di uscita (`hook_net_exit`, `_stat_exit`, `_msgget_exit`), il valore di ritorno della syscall viene estratto da `args[-1]` e normalizzato con segno su `ql.arch.bits` tramite la funzione helper `_signed(ql, value)`.
  5. I blocchi `except` silenziosi negli hook principali sono stati sostituiti dalla funzione `_hook_failed(name)`, che registra l'intero traceback a livello `DEBUG` all'interno di `complete_log`.

Sottosistema per il Debugging (`gdb_debugging_ext.py` / `emu_gdb.py`)

Colli di Bottiglia del Server GDB Nativo di Qiling e generale mancanza di feature QoL(`ql.debugger = "gdb"`)

- **Problema riscontrato:** Il server GDB predefinito di Qiling ha tre gravi limitazioni nell'emulazione di binari embedded multi-processo come `httpd`:

  1. *Lentezza in fase di inizializzazione:* L'esecuzione del dynamic linking, del polling sul socket NVRAM e del setup IPC SysV sotto lo stub GDB in Python risulta lenta al limite dell'inutilizzabile.
  2. *Assenza dei simboli delle librerie condivise:* Il server GDB di Qiling non implementa il protocollo di rendezvous del dynamic linker SVR4 (`r_debug`), lasciando `libc.so`, `libcfm.so` e le altre librerie condivise come indirizzi non mappati in GDB.
  3. *Collisioni di porta su `fork()`:* Poiché Qiling emula `fork()`/`vfork()` tramite chiamate reali a `os.fork()` sull'host, qualsiasi processo figlio eredita la porta TCP associata a GDB (`9999`), causando conflitti di socket o blocchi.

- **Origine del problema:** Lo stub `QlGdb` nativo è progettato per sessioni single-process statiche fin dall'entry point e non gestisce né la rilocazione automatica delle sezioni `.text` delle shared library caricate da `ql.loader`, né il multiplexing delle porte TCP dopo una `os.fork()`.

- **Soluzione implementata (`GdbManager` in `gdb_debugging_ext.py`):** Il modulo esegue l'emulatore alla massima velocità di Unicorn e avvia dinamicamente un server `QlGdb` fork-safe e consapevole dei simboli **esclusivamente al verificarsi di un evento target o di un memory fault**, controllato tramite variabili d'ambiente (attivo di default con `EMU_GDB=on` in `httpd_emulation_harness.py`):

  1. *Attivazione On-Demand basata su Eventi (Event-Triggered Spawner):* Invece di collegare GDB al boot, `httpd` si inizializza normalmente e la sessione GDB viene aperta da `GdbManager.spawn()` (prevenendo sessioni duplicate sulla stessa porta tramite il set `_active_sessions`) solo quando scatta uno dei seguenti trigger configurabili via environment:
     - `EMU_GDB_AT_ENTRY=1`: Registra un `ql.hook_address` su `ql.loader.elf_entry` per mettere in pausa l'esecuzione all'entry point ELF prima di `main()`.
     - `EMU_GDB_ON_URI`: Invocato da `hook_net_exit` (`check_http_trigger`), mette in pausa l'emulazione all'uscita di `recv`/`recvfrom`/`read` non appena viene intercettata una richiesta HTTP contenente la sottostringa specificata (es. `/goform/GetRouterStatus`).
     - `EMU_GDB_ON_EXEC`: Invocato da `hook_execve` (`check_exec_trigger`), mette in pausa l'esecuzione quando il percorso del binario o la stringa degli argomenti contengono la sottostringa indicata.
     - `EMU_GDB_ON_IPC`: Invocato da `_msgsnd_enter` (`check_ipc_trigger`), legge dalla memoria del guest il campo `mtype` e il payload del messaggio e mette in pausa l'emulazione se la stringa configurata compare nel payload o corrisponde a `str(mtype)`. In `httpd_emulation_harness.py`, `_msgsnd_enter` viene registrato con `QL_INTERCEPT.ENTER` anche quando `IpcShim` è attivo, così da preservare il trigger GDB senza interferire con la sostituzione della syscall.
  2. *Generazione Automatica dello Script di Caricamento Simboli (`logs/attach_<pid>.gdb`):* Ad ogni invocazione di `spawn()`, il metodo `generate_gdb_script()` ispeziona `ql.loader.images` e, per ogni file contenente `.so` nel nome, calcola l'offset virtuale (`sh_addr`) della sezione `.text` tramite la

funzione `_get_elf_text_offset(elf_path)` (che utilizza `pyelftools` se disponibile, oppure un parser binario di fallback basato su `struct.unpack_from` per header ELF a 32-bit little/big-endian). Genera quindi il file `logs/attach_<pid>.gdb`, che imposta l'architettura ARM, configura `solib-absolute-prefix` su `ROOTFS`, carica il binario principale con `file`, mappa ogni libreria condivisa all'indirizzo `img.base + text_off` tramite `add-symbol-file` ed esegue `target remote 127.0.0.1:<port>`, `info registers` e `x/5i $pc`.

3. *Trappola di Crash Just-In-Time (`EMU_GDB_ON_CRASH=1`) e Trace dump (`EMU_GDB_TRACE=1`):* Un hook sui basic block (`ql.hook_block(self._hook_block)`) mantiene una `deque` circolare (`trace_ring`, di profondità predefinita `EMU_GDB_TRACE_DEPTH=128`) contenente le tuple `(address, lr, sp)` lette da `ql.arch.regs.lr` e `ql.arch.regs.arch_sp`. Quando Unicorn rileva un accesso invalido o protetto alla memoria (`UC_MEM_READ_UNMAPPED`, `UC_MEM_WRITE_UNMAPPED`, `UC_MEM_FETCH_UNMAPPED`, `UC_MEM_WRITE_PROT`, `UC_MEM_READ_PROT`, `UC_MEM_FETCH_PROT`, codici 16–21) intercettato da `ql.hook_mem_invalid(self._hook_mem_invalid)`, `GdbManager` registra a livello `ERROR` l'indirizzo, il tipo di accesso e il valore coinvolto, invoca `dump_trace(ql)` per riversare nel logger `emu.hook.gdb` (`small_log` e `complete_log`) la trace con risoluzione simbolica `<immagine>+<offset>` e avvia il server GDB sull'istruzione che ha causato il fault prima di restituire `False`.
4. *Routing delle Porte Multi-Processo Fork-Aware:* Il metodo `GdbManager._get_port()` confronta `os.getpid()` con il PID iniziale salvato all'avvio (`self._initial_pid`): il processo padre `httpd` utilizza la porta base `EMU_GDB_PORT` (default `9999`), mentre qualsiasi processo figlio generato da `fork()` utilizza la porta deterministica `10000 + (os.getpid() % 5000)` e genera uno script `logs/attach_<child_pid>.gdb` isolato.
5. *Comandi GDB `monitor` Personalizzati:* Sostituendo il metodo `gdb.handle_qRcmd` sull'istanza `QlGdb`, il gestore decodifica il payload esadecimale dei comandi `monitor` inviati da `gdb-multiarch` ed esegue `_handle_monitor(ql, cmd_str)`:
   - `monitor help`: Elenca i comandi `monitor` disponibili.
   - `monitor vmmap`: Restituisce la mappa di memoria formattata di Qiling (`ql.mem.get_formatted_mapinfo()`).
   - `monitor trace [N]`: Mostra gli ultimi `N` (default 32) basic block registrati in `trace_ring`, risolvendo `PC` e `LR` nel formato `<immagine>+<offset>` tramite `_resolve_image_symbol` insieme a `SP`.
   - `monitor where <addr>`: Risolve un indirizzo esadecimale o intero nell'immagine ELF corrispondente (`<name>+<offset>` oppure `unmapped`).
   - `monitor ipc`: Legge il file di registro `logs/ipc_queues.txt` gestito da `IpcShim` per elencare le code SysV attive sull'host (`msqid key creator_pid`).
   - `monitor nvram list` / `monitor nvram set <key> <val>`: Elenca o aggiorna a caldo le coppie chiave/valore nel dizionario condiviso `NVRAM_STORE` utilizzato da `nvram_listener`.
   - `monitor snapshot save [name]` / `restore [name]` / `list`: Salva in memoria (`ql.save(reg=True, mem=True, cpu_context=True)`), ripristina (`ql.restore(...)`) o elenca gli snapshot dello stato CPU e RAM sotto la chiave `[name]` (default `default`).

Limitazioni Attuali

Assenza dei deamon riceventi e Semantica IPC Incompleta

`ipc_shim.py` permette a `httpd` e `cfm` di depositare con successo i messaggi nelle code del kernel host, evitando il blocco del mittente. Tuttavia, poiché i demoni di backend del firmware (es. `netctrl`, `time_check`) non sono in esecuzione, nessun processo consuma tali messaggi o applica le modifiche di configurazione richieste.

Al momento non sono state esplorate concretamente soluzioni ma, considerati gli obiettivi del progetto, sarebbe probabilmente sufficiente un modulo atto a consumare i segnali IPC strettamente necessari all'emulazione della funzionalità che si sta analizzando. Alternativamente ci sarebbe la possibilità di emulare separatamente il daemon attraverso il modulo attuale in quanto l'IPC è gestito attraverso le queque SystemV della macchina host. Ciò richiederebbe però un considerevole effort nel generalizzare le funzionaità presenti e la probabile implementazione di soluzioni custom (come già fatto per `httpd`) per problematiche del singolo deamon.

Assenza di reale multithreading

Qiling non supporta l'emulazione di più thread in parallelo ma la simula eseguendo i singoli thread secondo scheduling round-robin. Nel contesto applicativo di questo progetto questo approccio impedisce analisi conclusive su qualsiasi tipo di vulnerabilità basata su race condition.

Essendo una limitazione strutturale di Qiling l'unica soluzione è un'esecuzione realmente concorrente dei thread guest, ciò richiederebbe lo sviluppo di un nuovo motore di emulazione, ben al di fuori dello scope di questo progetto.